DevOps Learning Guide

Zero to Job Ready

A complete curriculum covering Linux, Git, Terraform, Kubernetes, CI/CD, cloud platforms, monitoring, security, and career preparation.

DevOps Learning Guide -- Zero to Job Ready

A complete, structured curriculum to go from knowing nothing about DevOps to being employable as a Junior DevOps Engineer. This guide assumes you already understand **Docker** basics; everything else is built from the ground up.

How to use this guide

Each **phase** represents a learning milestone. Within each phase, **topics** are ordered by dependency -- complete them in sequence. Every topic has a **do this** section with concrete actions. Don't just read; type every command, break every config, fix it, then move on.

Estimated timeline: 6-12 months of consistent part-time study (10-15 hrs/week).

Phase 0: The Prerequisites

Before touching any DevOps tool, you need actual computer fundamentals. These are the filters that screen out most candidates.

Linux Fundamentals

Know your way around a headless server. No GUI, no mouse.

- Filesystem hierarchy (`/`, `/etc`, `/var`, `/proc`, `/sys`, `/tmp`)
- File operations: `ls`, `cp`, `mv`, `rm`, `find`, `locate`
- Text processing: `grep`, `sed`, `awk`, `cut`, `sort`, `uniq`, `wc`
- Permissions: `chmod`, `chown`, `umask`, special bits (SUID, SGID, sticky)
- Process management: `ps`, `top`/`htop`, `kill`, `systemd` units
- Package management: `apt`/`yum`/`apk`
- Networking: `ip`, `ss`, `ping`, `traceroute`, `dig`, `curl`
- Systemd: writing service units, journalctl, timers
- Filesystem management: `df`, `du`, `mount`, `fstab`, LVM basics

Do this: Install Ubuntu Server in a VM (or use a cheap VPS). Break it. Fix it. Repeat until you can recover a system where you accidentally removed `libc6`.

Bash Scripting

The glue of everything DevOps.

- Variables, arrays, parameter expansion
- Conditionals (`if`, `case`) and loops (`for`, `while`)
- Functions and scope
- Exit codes and error handling (`set -euo pipefail`)
- Argument parsing (`$@`, `getopts`)
- Subshells, command substitution, process substitution
- `jq` for JSON manipulation

Do this: Write a backup script that compresses a directory, uploads to a remote server, and cleans up files older than 7 days. Include proper error handling and logging.

Git

Not just `add`/`commit`/`push`. Understand the model.

- Object model: blobs, trees, commits, tags
- Branching and merging strategies (Git Flow, trunk-based)
- Rebasing vs merging -- when each applies
- Interactive rebase (`rebase -i`) -- squash, reword, drop
- Cherry-pick, revert, reset (soft/mixed/hard)
- `bisect` for finding the commit that broke something
- Hooks (pre-commit, pre-push)
- Conventional commits

Do this: Intentionally create a messy branch with 10+ commits. Use interactive rebase to squash, reword, and reorder them into 3 clean commits. Then use `git bisect` to find where a bug was introduced.

Networking Fundamentals

You cannot manage infrastructure without understanding how data moves.

- OSI model (focus on layers 3, 4, 7)
- TCP vs UDP -- three-way handshake, connection states
- DNS -- record types (A, AAAA, CNAME, MX, TXT), resolution flow
- HTTP/HTTPS -- methods, status codes, headers, cookies, caching
- TLS -- handshake, certificates, CAs, SNI
- Load balancing concepts -- round-robin, least connections, sticky sessions
- Subnetting -- CIDR notation, public vs private IPs, NAT

Do this: Use `tcpdump` or `tshark` to capture a TLS handshake, then identify each step (ClientHello, ServerHello, Certificate, etc.) from the capture.

Phase 1: Version Control & Collaboration

Master the Git workflows used in real teams.

Git Platform Proficiency

Pick one platform (GitHub recommended for market share).

- PR/MR workflows -- code review, approvals, draft PRs
- Branch protection rules -- required checks, status gates
- Actions basics -- YAML syntax, workflow triggers, runners
- Repository management -- webhooks, deploy keys, secrets
- GitHub CLI (`gh`) for terminal-based workflow

Do this: Set up a repo with branch protection requiring PRs, passing status checks, and linear history. Automate linting in CI.

Phase 2: Infrastructure as Code (IaC)

Your config files are now the source of truth.

Terraform

The dominant IaC tool. HashiCorp Terraform (or OpenTofu).

- Core concepts: desired state, execution plan, resource graph
- HCL syntax -- blocks, arguments, expressions, functions
- Providers and resources
- State management -- local vs remote (S3/GCS + DynamoDB/BigQuery locking)
- Variables and outputs
- Modules -- writing, publishing, versioning
- Workspaces for environment separation
- `terraform workspace`, `import`, `refresh`, `state` subcommands
- Remote backends (S3, GCS, AzureRM, Terraform Cloud)
- Sentinel / OPA policy as code (intro)

Do this: Write Terraform that provisions a VPC, a subnet, an EC2 instance with a security group, and an S3 bucket. Use a module from the registry. Configure remote state.

Infrastructure Testing

- `terraform plan` in CI with review
- `terraform-compliance` or `terraform-sentinel` for policy checks
- `terratest` for integration tests
- `checkov` / `tfsec` / `trivy` for security scanning

Beyond Terraform: Alternatives

- **Pulumi** -- same concept, general-purpose languages (TypeScript, Python, Go)
- **Crossplane** -- Kubernetes-native IaC, control plane pattern
- **CDK for Terraform** -- Terraform via programming languages

Know about them. Deep-dive one if your target job uses it.

Phase 3: Container Orchestration -- Kubernetes

K8s is the non-negotiable skill for DevOps roles.

Core Concepts

- Architecture: control plane (API server, scheduler, controller manager, etcd) vs worker nodes (kubelet, kube-proxy, container runtime)
- Pods -- the atomic unit
- Workloads: Deployments, StatefulSets, DaemonSets, Jobs, CronJobs
- Services: ClusterIP, NodePort, LoadBalancer, ExternalName
- Ingress and ingress controllers

- ConfigMaps and Secrets
- Namespaces for isolation
- Resource requests and limits (CPU, memory)
- Probes: startup, readiness, liveness

Do this: Deploy a simple web app (nginx) using `kubectl run`, then recreate it with YAML manifests. Add a Service, then an Ingress. Scale it. Do a rolling update. Roll back.

Storage

- Volumes, PersistentVolumes, PersistentVolumeClaims
- StorageClasses and dynamic provisioning
- StatefulSets with volumeClaimTemplates
- CSI drivers

Do this: Deploy PostgreSQL as a StatefulSet with persistent storage. Scale it down and up. Verify data survives pod deletion.

Security

- RBAC -- Roles, ClusterRoles, RoleBindings, ClusterRoleBindings, ServiceAccounts
- Pod Security Standards / Pod Security Admission
- NetworkPolicies for pod-level segmentation
- Security contexts -- runAsNonRoot, readOnlyRootFilesystem, capabilities
- Seccomp profiles
- Secrets encryption at rest

Package Management (Helm)

- Charts: structure, templates, values
- Built-in objects (Release, Chart, Files, Capabilities)
- Template functions and pipelines
- Dependency management
- Repositories and publishing
- Helmfile for multi-environment releases

Do this: Create a Helm chart for the web app you deployed earlier. Parameterize the image tag, replicas, and resource limits.

Service Mesh (optional but differentiator)

- Istio or Linkerd basics
- Sidecar injection, mTLS, traffic splitting
- Observability (metrics, traces, access logs)

Do this: Deploy Istio on your cluster, enable mTLS, and route 10% of traffic to a canary version of your app.

Phase 4: Configuration Management & Automation

Ansible

Agentless, push-based, YAML-driven.

- Inventory (static and dynamic)
- Modules: `command`, `copy`, `template`, `service`, `apt`, `file`
- Playbooks -- plays, tasks, handlers, variables
- Roles and directory structure
- Jinja2 templating
- Ansible Vault for secrets
- Pull mode vs push mode

Do this: Write an Ansible playbook that provisions a web server -- install nginx, configure a virtual host, deploy an HTML file, open the firewall.

Other Tools (Know of Them)

- **Chef** / **Puppet** -- older, agent-based, still in many enterprises
- **Salt** -- fast, event-driven
- **Packer** -- machine image as code (AMI, GCE image, Vagrant box)

Phase 5: CI/CD Pipelines

Automate everything from commit to production.

CI/CD Concepts

- Continuous Integration: merge often, build and test every commit
- Continuous Delivery: artifacts always deployable
- Continuous Deployment: every commit that passes tests goes to production
- Pipeline stages: lint -> test -> build -> scan -> deploy
- Artifact management (registry, versioning)

GitHub Actions

The easiest to start with and widely used.

- Workflow syntax: `on`, `jobs`, `steps`, `runs-on`
- Actions marketplace -- reusable actions
- Self-hosted runners
- Environments, secrets, and approvals
- Matrix builds
- OIDC for cloud auth (no static creds)

Do this: Build a CI pipeline that lints, tests, builds a Docker image, pushes to Docker Hub, and deploys to a staging K8s cluster.

GitLab CI

- `.gitlab-ci.yml` -- stages, jobs, artifacts, caching

- Runners (shared, specific, group)
- Auto DevOps
- Review Apps (ephemeral environments per MR)

GitOps with ArgoCD

ArgoCD is the industry standard for Kubernetes GitOps.

- Application CRD, Sync policy, sync waves
- SSO integration (dex, OIDC)
- Image updater
- Argo Rollouts for progressive delivery (blue/green, canary)
- Multi-cluster management

Do this: Bootstrap an ArgoCD Application that syncs a deployment from a Git repository. Make a change in Git -- watch ArgoCD apply it automatically.

Other Tools (Know of Them)

- **Jenkins** -- still ubiquitous in enterprise. Groovy pipelines, shared libraries
- **CircleCI**, **Buildkite** -- SaaS alternatives
- **Flux** -- alternative GitOps operator

Phase 6: Cloud Platforms

Deep-dive on **one** cloud. Know the other two at conversation level.

AWS (largest market share)

- Core: EC2, S3, VPC, IAM, Route53
- Containers: ECS, EKS, ECR
- Databases: RDS, Aurora
- Networking: ALB/NLB, CloudFront, VPN
- Security: KMS, Secrets Manager, WAF, Shield
- Automation: CloudFormation, CDK, SSM
- Serverless: Lambda, API Gateway, EventBridge

GCP

- Core: Compute Engine, Cloud Storage, VPC, IAM, Cloud DNS
- Containers: GKE (best managed K8s experience), Artifact Registry
- Networking: Cloud Load Balancing, Cloud CDN, Cloud NAT
- Security: Cloud KMS, Secret Manager, Cloud Armor
- Automation: Deployment Manager, Cloud Build

Azure

- Core: VMs, Blob Storage, VNet, Entra ID
- Containers: AKS, ACR

- Networking: Azure Load Balancer, Front Door, Application Gateway
- Security: Key Vault, Defender for Cloud

Do this: Get a **certification**. This forces structured learning and looks good on a resume.

- AWS: Solutions Architect Associate (SAA-C03)
- GCP: Associate Cloud Engineer
- Azure: AZ-104

Phase 7: Monitoring & Observability

You cannot improve what you don't measure.

Metrics & Alerting

- **Prometheus**: data model, PromQL, exporters (node_exporter, kube-state-metrics), ServiceMonitor, recording rules, alerting rules
- **Grafana**: dashboards, panels, alerting, annotations
- Alertmanager: grouping, inhibition, routing, silences
- **PagerDuty** / **Opsgenie** for on-call

Do this: Set up kube-prometheus-stack (Prometheus + Grafana) on your K8s cluster. Create a dashboard showing CPU/memory/disk across all nodes. Set an alert for >80% disk usage that fires to a webhook.

Logging

- **EFK/ELK stack**: Elasticsearch, Fluentd/Logstash, Kibana
- **Loki**: Prometheus-inspired, horizontally scalable, cheap
- **OpenSearch**: AWS fork of Elasticsearch

Do this: Deploy Loki + Promtail on your cluster. Feed container logs to Grafana. Create a log-based alert.

Tracing

- **OpenTelemetry** -- the unified standard (traces + metrics + logs)
- Jaeger / Tempo for trace storage
- Distributed tracing concepts: spans, trace context, sampling

Do this: Instrument a simple Go app with OpenTelemetry, send traces to Jaeger, and visualize a request that spans two services.

SLOs and Error Budgets

- SLI, SLO, SLA definitions
- Error budget policy
- Burn rate alerting
- Multi-window, multi-burn-rate approach

Phase 8: Security (DevSecOps)

Security is not a phase -- it's embedded in every phase above. Call it out explicitly here.

Shift-Left Security

- **SAST**: Semgrep, CodeQL, SonarQube -- find bugs before commit
- **DAST**: OWASP ZAP, Burp Suite -- test running applications
- **SCA**: Trivy, Grype, Snyk -- scan dependencies for known CVEs
- **Container scanning**: Trivy, Docker Scout, Anchore
- **IaC scanning**: Checkov, tfsec, Trivy

Supply Chain Security

- **SLSA** framework (Supply-chain Levels for Software Artifacts)
- **SBOM** generation (Syft, Trivy)
- **Sigstore** / **Cosign** for signing containers
- **in-toto** attestations

Secrets Management

- **HashiCorp Vault**: dynamic secrets, transit engine, KV store, K8s auth
- **External Secrets Operator** / **Secrets Store CSI Driver** for K8s
- **Mozilla SOPS** + age/GPG for encrypted files in Git

Do this: Deploy Vault on your cluster. Configure Kubernetes auth. Write a controller that reads a DB password from Vault and injects it into pods.

Runtime Security

- **Falco** -- behavioral activity monitoring (the K8s security sibling of Sysdig)
- **OPA/Gatekeeper** -- admission controller for policy enforcement
- **Kyverno** -- Kubernetes-native policy engine

Phase 9: Real Projects (Build a Portfolio)

Theory is worthless without demonstrated work. Build these end-to-end and put them on GitHub.

Project 1: Automated Deployment Pipeline

- Source code in GitHub
- CI pipeline (GitHub Actions): lint -> test -> build -> scan -> publish image
- GitOps deploy (ArgoCD): auto-sync to dev, manual promotion to prod
- Infrastructure (Terraform): VPC, EKS cluster, node groups
- Monitoring: Prometheus metrics, Grafana dashboard, Loki logs
- Result: you commit, CI builds, ArgoCD deploys. End to end.

Project 2: Multi-Tier Application on K8s

- Frontend: nginx serving React/Vue/Astro static build
- Backend API: Go or Node.js
- Database: PostgreSQL StatefulSet
- Backup: CronJob that pg_dumps to S3/GCS
- Ingress with TLS (cert-manager + LetsEncrypt)
- NetworkPolicy restricting pod-to-pod traffic

Project 3: Disaster Recovery Simulation

- Terraform provisions infra in two regions
- Application runs in primary region
- Script that simulates primary failure (terraform destroy partial)
- Automate failover to secondary region
- Test RTO and RPO

Project 4: Kubernetes Operator (advanced)

- Write a custom operator in Go using `controller-runtime`
- Watch a custom CRD
- Reconcile desired state
- Deploy with OLM

Phase 10: Soft Skills & Career

System Design for DevOps

Common interview scenarios:

- Design a CI/CD pipeline for a microservices architecture
- Design a highly available K8s cluster across availability zones
- Design a logging infrastructure for 500 microservices
- Design a multi-region deployment strategy
- Design secrets management for 100+ engineers

Framework: understand requirements -> sketch data flow -> identify failure modes -> propose solution -> discuss tradeoffs.

Behavioral Questions

- "Tell me about a time an incident happened and how you handled it"
-- Use STAR: Situation, Task, Action, Result
- "How do you handle on-call and burnout?"
- "Tell me about a time you automated something that was manual"
- "How do you stay up to date with industry changes?"

Resume Tips

- Quantify impact: "Reduced deployment time from 45min to 8min"

- List tools, but more importantly what you achieved with them
- GitHub link with real projects beats "3 years experience" every time
- Include CI/CD badges in your project READMEs

Certifications Path (Optional but Helpful)

Cert | Provider | When
Linux Essentials / LPIC-1 | LPI | Before or during Phase 1
CKA (Certified Kubernetes Administrator) | CNCF | After Phase 3
AWS SAA (Solutions Architect Associate) | AWS | During Phase 6
CKAD (Certified Kubernetes App Developer) | CNCF | After CKA
Terraform Associate | HashiCorp | After Phase 2.1

CKA is the highest-value for DevOps roles. Do not skip it.

Interview Prep Checklist

Before applying, confirm you can:

- [ ] Explain what happens when you type `kubectl create deploy nginx --image=nginx`
(API call -> etcd write -> scheduler -> kubelet -> container runtime -> pod running)
- [ ] Debug a pod that's stuck in `CrashLoopBackOff` or `Pending`
- [ ] Write a Dockerfile that produces a small, secure image
- [ ] Write a Terraform config that provisions infrastructure with remote state
- [ ] Set up a CI pipeline that builds, scans, and deploys
- [ ] Explain the difference between a Deployment and a StatefulSet
- [ ] Explain how `terraform plan` works internally
- [ ] Describe how you'd migrate a 500-node cluster from one K8s version to the next without downtime

---

Recommended Resources

Books

- **The DevOps Handbook** -- Gene Kim et al. (cultural foundation)
- **Site Reliability Engineering** -- Google (SRE principles)
- **Kubernetes in Action** -- Marko Luk a (best K8s book)
- **Terraform: Up & Running** -- Yevgeniy Brikman
- **The Phoenix Project** -- Gene Kim (novel, but explains DevOps culture)

Free Courses

- [KodeKloud](https://learn.kodekloud.com) -- hands-on labs, great for K8s
- [Killercoda](https://killercoda.com) -- free interactive scenarios
- [Play with Kubernetes](https://labs.play-with-k8s.com) -- free K8s sandbox
- [Terraform Up & Running workshop](https://github.com/brikis98/terraform-up-and-running-code)
- [Awesome DevOps](https://github.com/awesome-devops/awesome-devops)

Certification Prep

- [Killer.sh](https://killer.sh) -- CKA/CKAD simulator (best mock exams)
- [ExamPro](https://www.exampro.co) -- AWS cert prep
- [Terraform Certification Guide](https://github.com/affinidi/terraform-associate-certification)

Practice Platforms

- [Killercoda](https://killercoda.com) -- scenarios on demand
- [Instruqt](https://instruqt.com) -- hands-on tracks
- [Katacoda (archived but great content)](https://www.katacoda.com)
- [Play with Docker](https://labs.play-with-docker.com)

Communities

- [r/devops](https://reddit.com/r/devops) -- real discussions
- [r/kubernetes](https://reddit.com/r/kubernetes)
- [DevOps Discord servers](https://discord.com/invite/devops)
- [CNCF Slack](https://slack.cncf.io)

Quick Reference: Weekly Study Plan

Week | Focus | Practical
1-2 | Linux fundamentals | Set up a server, navigate, process text
3 | Bash scripting | Write automation scripts
4 | Git deep-dive | Rebase, bisect, hooks
5-6 | Terraform | Provision infrastructure
7-10 | Kubernetes core | Deploy apps, services, ingress
11-12 | Helm + K8s storage | Charts, StatefulSets
13-14 | K8s security | RBAC, NetworkPolicies, Pod Security
15-16 | Ansible | Configuration management
17-18 | CI/CD (GitHub Actions + ArgoCD) | Full pipeline
19-22 | Cloud platform (pick one) | Deep study + certification
23-24 | Monitoring | Prometheus, Grafana, Loki
25 | Security basics | SAST, container scanning
26-28 | Project 1: automated pipeline | Build end-to-end
29-32 | Project 2: multi-tier app on K8s | Build end-to-end
33-36 | Advanced: operator, service mesh | Differentiate
37+ | Interview prep + job search | System design, behavioral

---

Remember: DevOps is a culture and practice, not a tool set. A senior DevOps engineer who knows Bash, Linux, and networking fundamentals will outperform a junior who has watched 10 hours of "Kubernetes CRASH course" videos. Build real things. Break them. Fix them. That's the entire curriculum.